1. What is REDIS and Why it Exist?

Wednesday, 16 September 2026 2:29 PM

Consider this simple product lookup API:

```
@GetMapping("/products/{id}")
public ProductDto getProduct(@PathVariable Long id) {
    return productRepository.findById(id)
        .map(ProductMapper::toDto)
        .orElseThrow(() -> new ProductNotFoundException(id));
}
```

This code is fine. Nothing wrong with it as long as traffic is low.

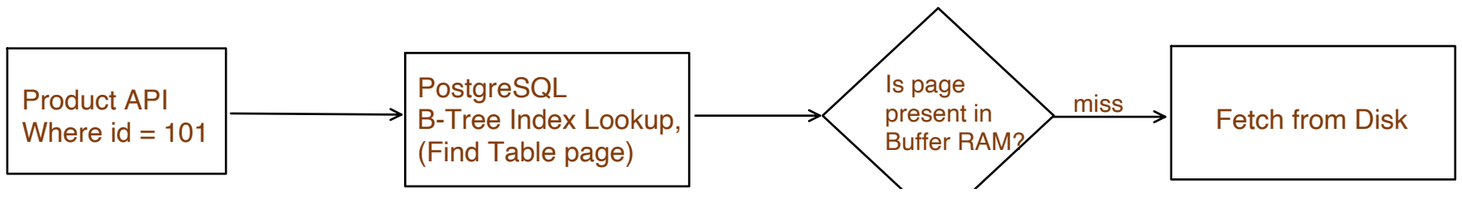
Now imagine this:
A sale goes live. One product (***id=101***) suddenly gets 50,000 requests in 2 minutes.
Every single one of those requests runs the exact same query:

SELECT * FROM products WHERE id = 101;

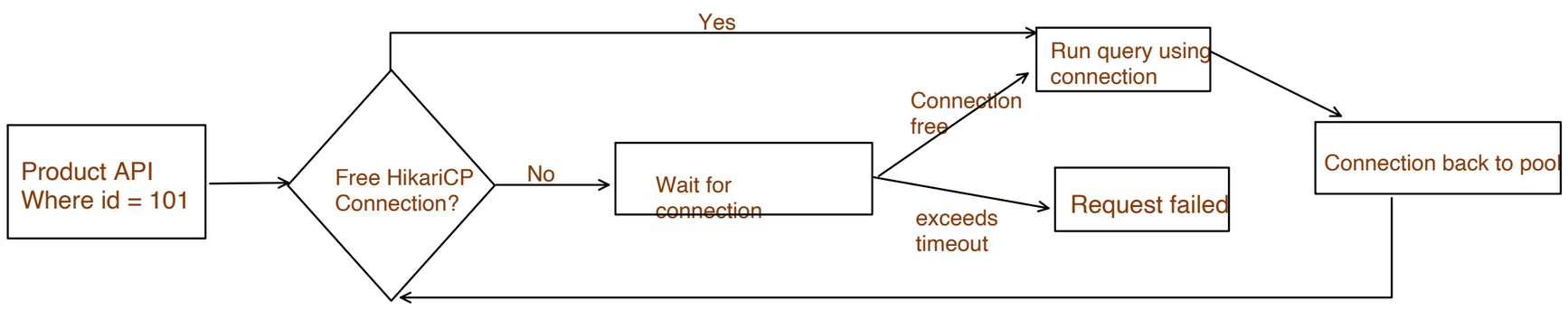
Problems:

- Every single time database has to go through index lookups, buffer checks and execute query.

In PostgreSQL, table data is stored in form of pages. And it also buffer pages which accessed frequently.



- A typical connection pool (like HikariCP) only has 10 - 20 connections per service instance. 50,000 requests trying to use 10-20 connections means massive queueing.



- The product data for id=101 hasn't changed in the last 5 minutes. But we are still asking the database to fetch it, 50,000 times, from scratch.
- We can scale vertically by paying more. But we are spending money to solve a problem that isn't really about capacity - it's about repetition.

REDIS can solve the above problem.

REDIS: REmote DIctionary Server

In simple words: Its an in-memory, key-value data structure store. Which can be accessed as a server.

in-memory:
It means: Redis keeps its data in RAM, not on a hard disk or SSD.

Note:
Redis does have an option to also save data to disk, so we don't lose everything on a restart. We'll cover that later.

Key-Value:

It means: Every piece of data in Redis is stored against a unique key, and we get it back using that same key. There's no table, no rows, no columns.

Example:
SET user:42:name "Rahul"
GET user:42:name

Output → "Rahul"

Why it matters:
There's no query planner, no scanning through rows, no searching. If we know the key, Redis goes straight to the value. This is a big part of why Redis operations are so predictable and fast.

Data Structure store:

This is the part that makes Redis different from a plain key-value cache (like ***Memcached***).

It means: A Redis "value" isn't just limited to plain text. It can be a proper data structure:

- List
- Hash
- Set
- Sorted Set

  etc.

And Redis lets us operate on these structures directly, without pulling the whole thing out first.

This is one of the biggest advantages of REDIS

Traditional storage follows Pull -> Modify -> Push :

MySQL flow:

- Application will fetch the complete row.
  Select * from User where id=101
- Deserialize to object and change 1 field say "age"
- Write modified row back

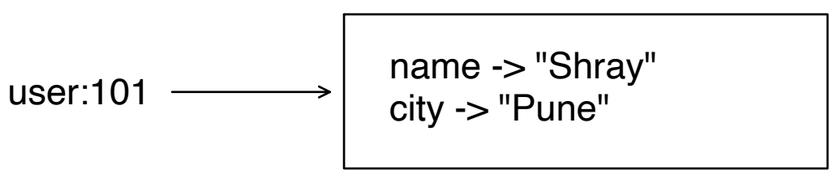
Memcached flow:

- Using Key, read the value, which is stored as plain blob (sequence of bytes).
- Deserialize and change 1 field say "age"
- Serialize it back and update the key value.

But Redis understand the structure of the Value, and thus let us update the object in-place.

Example:

HASH data structure:



Read only 1 field: HGET user:101 city
Output: Jaipur

Update only 1 field: HSET user:101 city "Bangalore"
Only city field changes

Similarly for other data structure like List, Set, Sorted Set etc. we can achieve this.

Server:

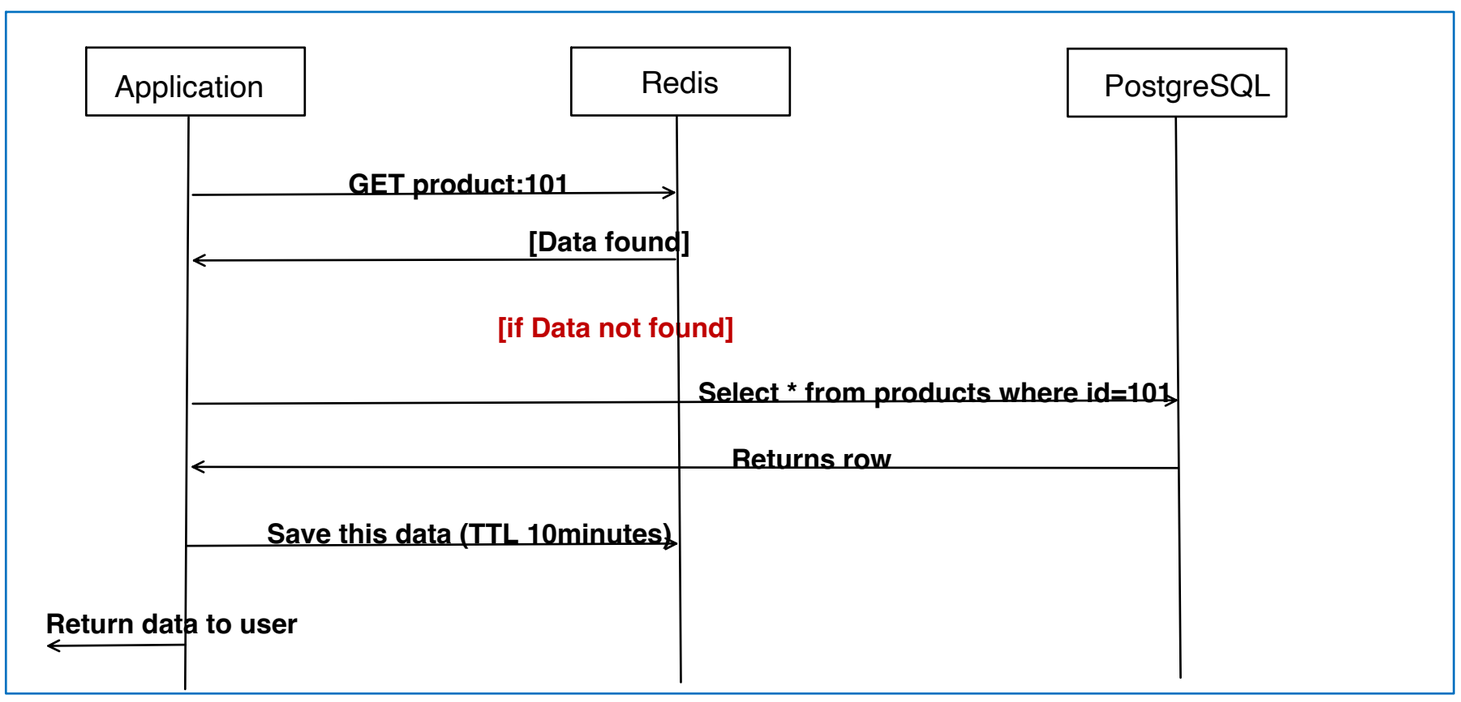
It means:
Redis is not a library we import into our code.
Our your application talks to it over the network (TCP), the same way it talks to a database.

Why it matters:
Because Redis is a separate server, every instance of our application can all talk to the *same* Redis server. That means they all see the same data.

Redis Use-cases:

1st: As a Cache

Pattern: cache-aside



Assume:

- 50,000 requests/minute for product:101
- A database query takes about 15ms
- A Redis lookup takes about 0.5ms

Without caching: 50,000 × 15ms = 750 seconds of total database time, every minute, just for one product.

With caching say only about 500 requests/minute reach the database -> around 7.5 seconds of database time per minute.

That's roughly a 100x drop in database load, just by not repeating the same work.

Cache Stampede

when a cached key expires, if thousands of requests arrive at almost the exact same moment, they can *all* miss the cache together and hit the database at once. This is called a Cache Stampede.

We will see later, how to handle this scenario to avoid the Cache Stampede.

2nd: As a primary database

In this use case, Redis isn't sitting *in front of* another database. For particular(not all) piece of data, Redis is the only place it lives.

Example1 : Storing a user session
- Temporary (expires after some time)
- Even if lost, user can create new session again (non critical)
- No complex query required

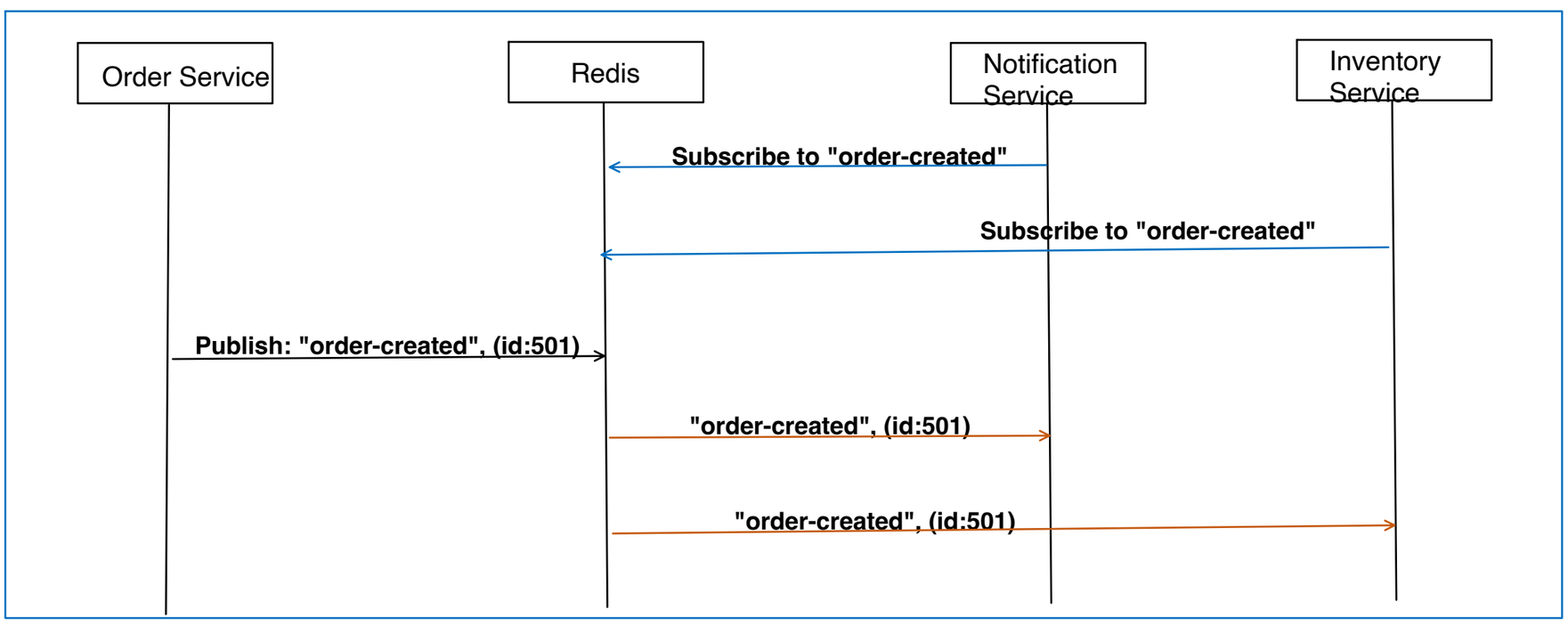
Example2: Real time gaming application
- Extremely high read/write frequency
- Low latency requirement
- Key based access
- Frequent atomic updates

Etc.

if losing data is not acceptable, then we need appropriate durability and availability configuration such as:
- Persistence - survives restart
- Replication - survives node failure

etc.

3rd: As a Message Broker



- Without Pub/Sub, Notification Service would have to keep asking, "any new orders yet?" every few seconds. That wastes of resources.
  With Pub/Sub, the moment an order is created, every subscriber hears about it instantly.

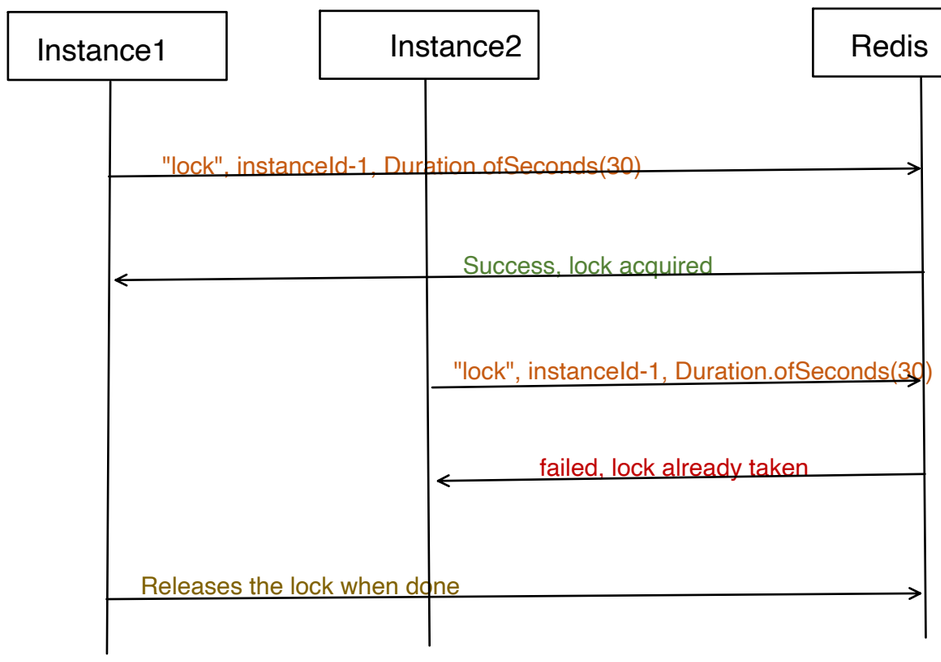
- Redis Pub/Sub is "fire and forget". If the Inventory Service happens to be down and at the exact moment a message is published, that message is lost forever. There is no queue holding it, no way to replay it later.

  This is very different from tools like Kafka, which store messages safely until they're processed.

4th: As a Distributed Coordination tool

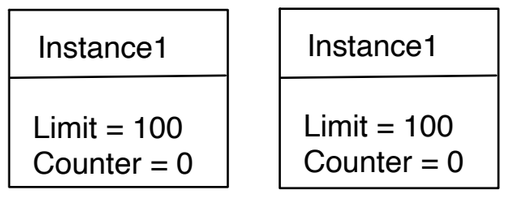
Distributed Locking

make sure only one instance of your app runs a scheduled job, even if you're running several copies of the app.

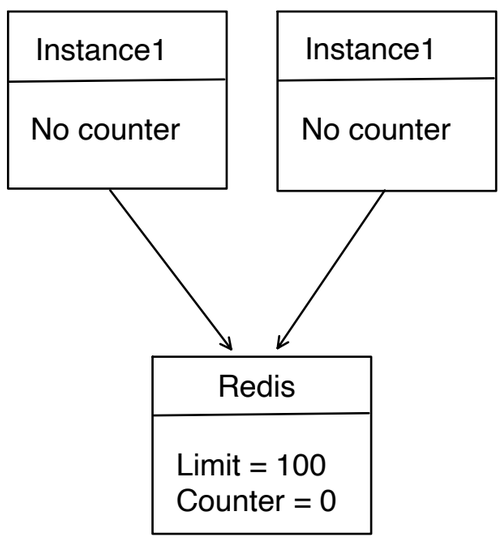


Rate Limiting

limit how many requests one user can make, across *all* our app instances combined.



Without Redis, user has 200 limit instead of 100.
Because each instance maintains their own counter.



Redis provides a shared, centralized counter that all application instances can atomically update.